HEALTHY KIDS

SUPPORT DOCUMENT

VERSION 1.1

TA08

1.0 Introduction

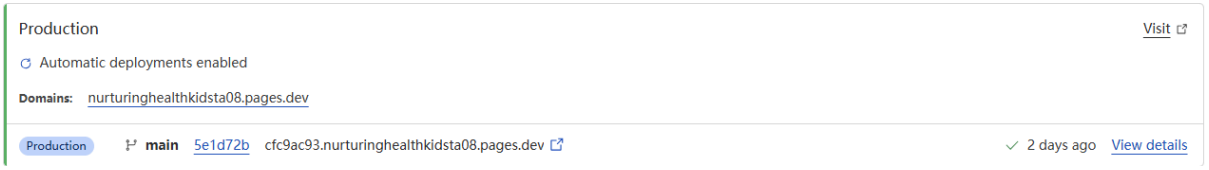
Our project Nurturing Healthy Kids is a web-based platform aimed at promoting physical activity, healthy eating, and lifestyle awareness among Australian children and their parents. This document is intended to guide sponsors, future developers, and system administrators to independently deploy, maintain, and extend the system infrastructure across cloud platforms and model APIs.

We recognize the importance of reliable, scalable, and secure digital solutions in public health interventions. This support document details the full-stack architecture setup, deployment processes, and best practices for managing the platform.

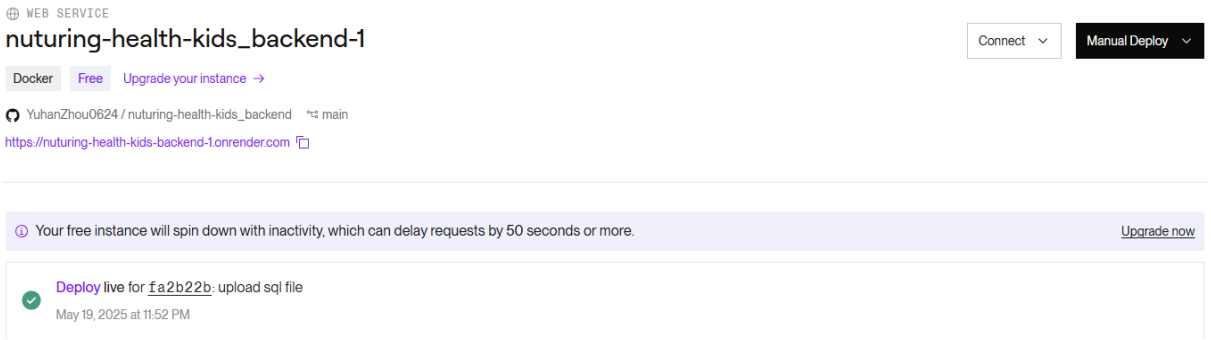
2.0 Support

Technical Overview

The system adopts a decoupled architecture, composed of:
- Frontend: Vue.js (deployed via Cloudflare Pages)



- Backend: FastAPI (hosted on Render)



- Database: Microsoft SQL Server (Azure cloud)
- Image recognition model: YOLOv5 model hosted on Hugging Face Spaces

The frontend interacts with the backend via RESTful API calls, while the backend communicates with both the database and the YOLO model endpoint.

2.1 Frontend-Cloudflare Pages

- Framework: Vue 3 + Vite
- Deployment platform: Cloudflare Pages (automatic deployment from GitHub)
- Main URL: https://nurturinghealthkidsta08.pages.dev/
- Deployment steps:
  1. Code push to main branch on GitHub

  2. Cloudflare CI builds via npm run build
  3. Files hosted under /dist
- Environment configuration:
  - API base URL stored in .env.production
  - Example:
VITE_API_BASE_URL=https://nuturing-health-kids-backend-1.onrender.com

2.2 Backend – Render (FastAPI)

- Framework: FastAPI (Python 3.10)
- Hosting: Render Web Service
- Service URL: https://nuturing-health-kids-backend-1.onrender.com
- Startup command: uvicorn app.main:app --reload
- Dependencies: pip install -r requirements.txt
- API Functionality Includes:
  - Guideline by age
  - Nearby park retrieval
  - Swap food quiz interface
  - Nutrition planner and AI integration

2.3 YOLO Model Integration – Hugging Face

- Model type: YOLOv5, custom-trained
- Hosting: Hugging Face Spaces or Inference API
- Backend Usage:
  - Requests sent using Python requests or https
  - Input: base64 image / file upload
  - Output: dish name, ingredients, steps
- Example call:
  response = requests.post("https://hf.space/api/predict", json={"image": encoded_img})

2.4 Database – Azure SQL Server

- Database: SQL Server (Azure PaaS)
- Connection string:

DRIVER="mssql+pyodbc://unionparty:monash101%21@ie-project.database.windows.net/iteration-1?driver=ODBC+Driver+18+for+SQL+Server&Encrypt=yes&TrustServerCertificate=no"
PWD=monash101!
Backend ORM: SQLAlchemy + pyodbc
Tables:

- melbourne_playgrounds
- guidelines_summary
- recipe_table
- Nurtition_guidelines

- Game

2.5 Environment Setup Summary

Component	Command / Tool
Vue Frontend	npm install && npm run build
FastAPI Server	pip install -r requirements.txt + gunicorn
Model	External API (no deployment required)
SQL Server	Managed in Azure Portal + pyodbc connection

3.0 Backup

- Source code version control: GitHub private repository
- Backup strategy:
- Git-based: Full change history retained
- Manual deployment snapshot: zip -r frontend-$(date).zip dist/
- Azure SQL Database: Automatic backup enabled by Azure (retention: 7–35 days)
- Render: Deploy logs accessible for rollback via commit hash

4.0 Security

4.1 Domain & HTTPS

- Cloudflare SSL automatically applies to all custom domains
- HTTP to HTTPS enforced using:
- Always Use HTTPS: true
- Strict-Transport-Security headers via Cloudflare Rules

4.2 Third-Party Security Testing

- Use OWASP ZAP, SecurityHeaders.com, or Burp Suite for vulnerability scanning
- Review CORS headers, API key exposure, and input sanitization
- Monitor Render logs for suspicious access attempts

4.3 Application Layer Security

- CORS Restriction:

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
        CORSMiddleware,
    allow_origins=["https://nurturinghealthkidsta08.cloudflarepages.dev"],
        allow_credentials=True,
```

```
        allow_methods=["*"],
        allow_headers=["*"],
)
```

- Disable docs in production:
  app = FastAPI(docs_url=None, redoc_url=None)

- SQL Injection protection:
  - Use SQLAlchemy ORM
  - Avoid raw SQL queries

5.0 Data Management

5.1 Database Setup

We have used Azure Database. Refer to below document: Database Setup

5.2 Open Source Data

Open source:

Data Sources					
Names	**Physical access**	**Frequency of source updates**	**Frequency of iteration system updates**	**Granularity**	**Copyright/ licensing details**
City of Melbourne Open Data	CSV	Last modified 13 Nov 2022 (irregular)	TBD	High	CC BY Attribution required. Source: data.melbourne.

					vic.gov.au
City of Casey Open Dataset	CSV	Last Updated 18 November 2024	Last Updated 9 February 2025	Suburb, postcode, latitude, longitude, area, map reference	https://data.casey.vic.gov.au/explore/dataset/playgrounds/table/
Physical Activity Guidelines	Available online via the Department of Health and Aged Care website	Last updated on 7 May 2021	No last update mentioned.	Guidelines are categorized by age groups: infants (birth to 5 years), children and young people (5 to 17 years), adults (18 to 64 years), older adults (65 years	Commonwealth of Australia, Department of Health and Aged Care

				and over), and specific recommendations for pregnancy and disability	
RecipeNLG Dataset	CSV	Not updated frequently	Manually as needed	High	Non-commercial, research and educational use only – Kaggle Dataset
Nutrition Guidelines for Children and Adolescents	Manually created CSV from website	Rarely updated	Manually as needed	Medium (age + gender)	CC BY 4.0 – Source: Australian Government, Eat for Health

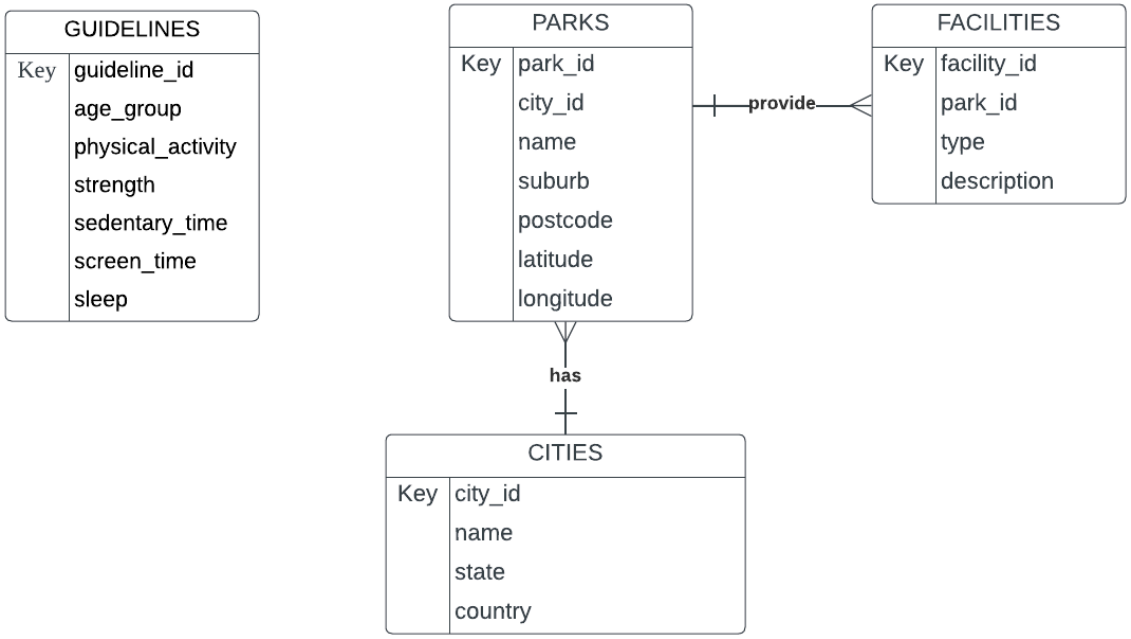
Australian National Health Survey – Weight status by age & sex (2022 cross-section)	CSV	Collected every 2–3 years; last fielded 2022	TBD	High	Australian Bureau of Statistics, released under CC BY 4.0
Australian National Health Survey – Physical-activity sufficiency (2011-12)	CSV	Irregular special module; last modified Nov 2013	TBD	High	Australian Bureau of Statistics, CC BY 4.0
Australian National Health Survey – Weight status trend 1995-2022	CSV	Updated after each survey wave (1995, 2007-08, 2011-12, 2014-15, 2017-18, 2022)	TBD	Medium	Australian Bureau of Statistics, CC BY 4.0
Australian Food Composition Database – Food Details	Excel (downloaded from foodstandards.gov.au)	Infrequently updated by FSANZ (last: 2021)	Updated once per major game data refresh	Individual food items	CC BY 4.0 – Attribution required

					(FSANZ)
Australian Food Composition Database – Nutrient File	Excel (downloaded from foodstandards.gov.au)	Infrequently updated by FSANZ (last: 2021)	Updated once per major game data refresh	Nutrients per 100g of food	CC BY 4.0 – Attribution required (FSANZ)
Food Images (Food-101)	Image files (JPEG/PNG)	Static dataset (no updates since initial release)	Model retrained quarterly	Medium	Data files © Original Authors from Kaggle
Food.com Recipes and Interactions	CSV	Rarely updated	Manually as needed	High	Publicly crawled data; redistributed under Food.com Terms of Use. Anonymized user interaction data.

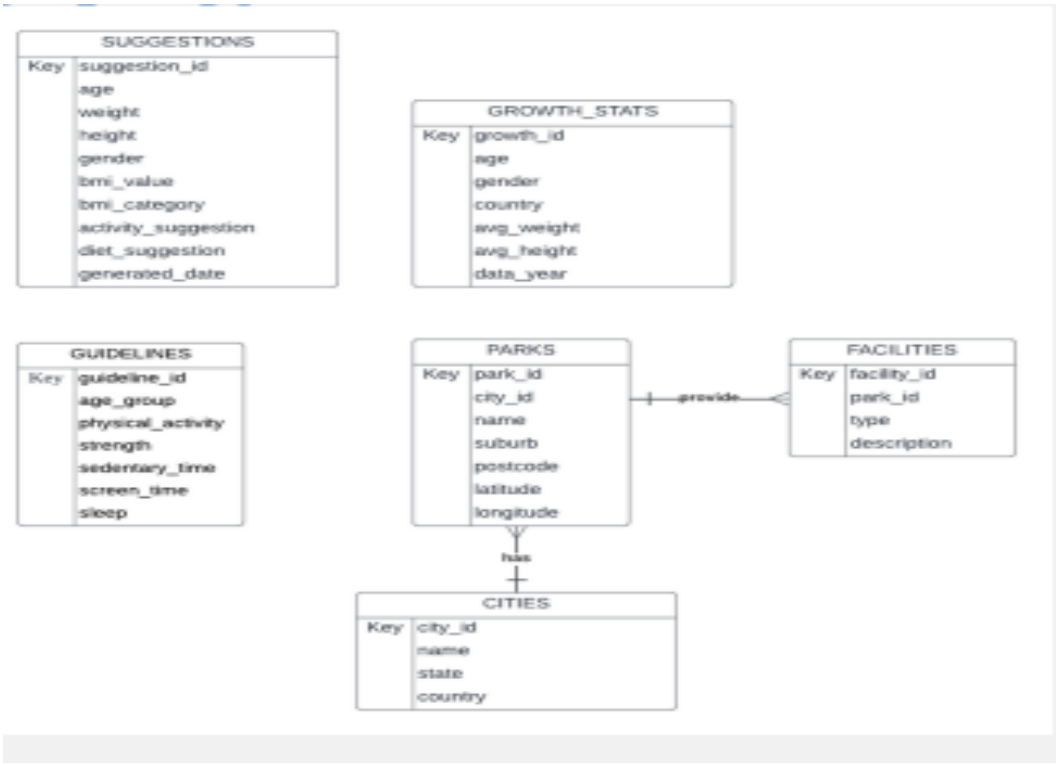
					from Kaggle

Entity Relationship Diagram

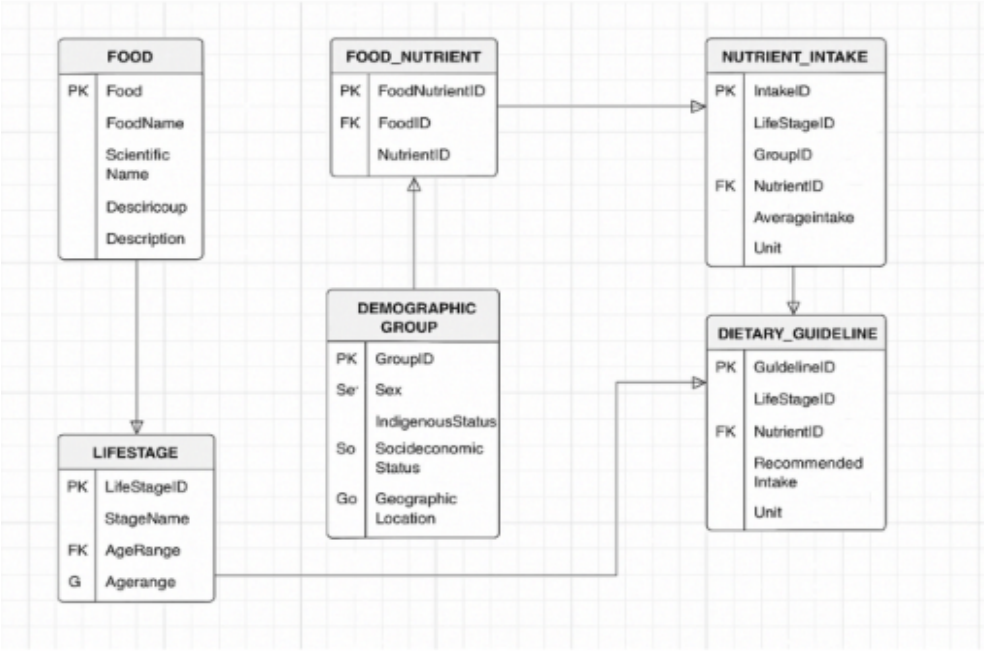
Iteration 1



Iteration 2


SUGGESTIONS
Key
suggestion_id
age
weight
height
gender
bmi_value
bmi_category
activity_suggestion
diet_suggestion
generated_date
GROWTH_STATS
Key
growth_id
age
gender
country
avg_weight
avg_height
data_year
GUIDELINES
Key
guideline_id
age_group
physical_activity
strength
sedentary_time
screen_time
sleep
PARKS
Key
park_id
city_id
name
suburb
postcode
latitude
longitude
provide
FACILITIES
Key
facility_id
park_id
type
description
has
CITIES
Key
city_id
name
state
country

Iteration 3


FOOD
PK
Food
FoodName
Scientific Name
Descriptoup
Description
FOOD_NUTRIENT
PK
FoodNutrientID
FK
FoodID
NutrientID
NUTRIENT_INTAKE
PK
IntakeID
LifeStageID
GroupID
FK
NutrientID
Averageintake
Unit
DEMOGRAPHIC GROUP
PK
GroupID
Se
Sex
IndigenousStatus
So
Socideconomic Status
Go
Geographic Location
DIETARY_GUIDELINE
PK
GuidelineID
LifeStageID
FK
NutrientID
Recommended Intake
Unit
LIFESTAGE
PK
LifeStageID
StageName
FK
AgeRange
G
Agerange

5.3 Data Preparation

Iteration 1:

- **City of Casey:**

The original dataset from the City of Casey facilities spreadsheet is processed and cleaned in this notebook. First, the raw data had inconsistent feature names (e.g., "Netball", "Cricketnet", "B/ball"), duplicate entries for the same physical locations, and no standard formatting.

In order to solve this, we first eliminate duplicates using a special name, latitude, longitude, and postcode combination. Then, for each site, we combined the features into a string separated by commas after grouping them by location. To increase clarity and uniformity, we then standardized the feature descriptions using more accessible terminology like "Basketball court," "Cricket net," and "Netball court." Finally, we added a placeholder for address data that can be updated with external lookup information or accurate geolocation in the future.

Link:https://colab.research.google.com/drive/1V_S9f5HK7n3GwMdIgPWREo5OsgD_cfz8?usp=drive_link

- **Melbourne_playground_clean**: This notebook performs data cleaning and preparation on the raw Melbourne playground dataset (playgrounds_melb.csv) to generate a structured and clean version (playgrounds_melb_cleaned_final.csv) for use in our website. The cleaned dataset allows the web platform to access reliable and well-formatted playground information, enabling users to view maps, filter parks by features, and retrieve relevant data based on location. Processing Steps include:
    - Remove columns whose names start with 'Unnamed', typically unwanted index columns from the original file.
    - Drop fully empty rows to eliminate blank or corrupted records.
    - Manually remove problematic rows at index 38 and 39, which were identified as structurally inconsistent.
    - Retain only essential columns: Geo Point, Geo Shape, council_re, features, and name.
    - Split the GeoPoint string into separate Latitude and Longitude columns for geospatial processing.

Link:https://drive.google.com/file/d/1qGZEeBmWAnx5P2doU5GQ4jyr97OUIzV_/view?usp=drive_link

Iteration 2:

- Playgrounds and recreational facilities located around the City of Casey are featured in this dataset, which has been cleaned and enhanced. At first, the raw data lacked address-based or graphic upgrades, had inconsistent naming rules, and contained duplicate entries for the same locations.
  The following were important cleaning and enriching steps:
  - Deduplication of records based on distinct names, latitudes, and longitudes.
  - Features are standardized to make them easier to read (for example, "B/ball" is changed to "Basketball court").
  - Park_address is formatted and cleaned for uniformity and GIS ready.
  - Curation of Image URLs by Hand: For every playground, each image_url was manually added by locating pertinent and openly accessible photographs on the internet. In order to guarantee visual enrichment for upcoming apps like dashboards, maps, or community interaction tools, this step was crucial.

  Link: Playground.xlsx

- Visualization :
  This notebook prepares three cleaned and filtered datasets from two large official Excel sources published by AIHW. The extracted datasets cover (1) obesity rates in children aged 2–17, (2) national obesity trends from 1995 to 2022, and (3) physical activity levels across age and gender groups. The cleaning process involves filtering by relevant age groups, selecting specific columns such as sex, year and saving the results into three compact CSV files. These cleaned files are designed to support our web product's interactive D3.js visualisations, enabling users to explore health and physical activity trends in an intuitive and informative way. Processing Steps include:
  - Load and parse the relevant sheets from two large Excel workbooks.
  - Extract age-specific obesity data by filtering age group values such as "2–4", "5–17", and "2–17" from Table S4.
  - Select only key columns such as Year, Age group, Sex, Obesity (%), Sufficient activity (%), etc.
  - Save each cleaned dataset into a dedicated CSV file ready to be loaded by D3-based charts on the website.

  https://drive.google.com/file/d/13yQ9MfZgVFeZdA-WuniwmzmArc6HZCOo/view?usp=drive_link

- Nutrition Planner:

  1. Data Loading & Initial Cleaning We transformed the original RecipeNLG dataset containing 501 recipes from a basic collection with 5 columns (title, ingredients, directions, link, NER) into a comprehensive meal planning dataset. The preprocessing pipeline began with extensive data cleaning: converting string-formatted lists like "['flour', 'sugar']" to actual Python lists for processing,

implementing fallback parsing for malformed data, removing recipes with missing essential information, and standardizing column names from 'title' to 'food_name' and 'NER' to 'main_ingredigents' for database compatibility.

2. Dietary & Meal Type Classification We engineered sophisticated dietary categories by analyzing ingredient lists for specific keywords to detect meat/fish (beef, chicken, pork, fish), dairy products (milk, cheese, butter, cream), and gluten-containing items (flour, bread, wheat, pasta), then classifying recipes as Non-Vegetarian, Vegetarian, or detailed combinations like "Dairy-Free, Gluten-Free, Vegetarian" to support various dietary restrictions. Meal types were determined through keyword matching on recipe titles using patterns like pancake/muffin/omelet→Breakfast, cake/cookie/chocolate→Dessert, dip/chip/ball→Snack, soup/salad→Lunch, with everything else defaulting to Dinner.

3. Time & Budget Categorization Cooking time categories were created by extracting time mentions from cooking directions using regex pattern matching for phrases like "bake for 30 minutes" or "cook for 1 hour", converting all units to minutes, summing multiple time references, and categorizing into "Less than 15 mins", "15-30 mins", or "More than 30 mins". Budget categories utilized comprehensive Australian dollar pricing research to estimate total recipe costs by matching ingredients to a cost database (proteins: chicken $2.20, beef $3.00, salmon $4.50; basics: eggs $0.40, flour $0.45), classifying as "Under $5" (<$7.50 AUD), "$5-10" ($7.50-15), or "Over $10" (>$15) to reflect realistic Australian grocery shopping.

4. Food Groups & Quality Assurance Food groups were identified by detecting multiple nutritional categories (protein, dairy, grains, vegetables, fruit, discretionary foods) within ingredient lists and intelligently combining them with comma separation (e.g., "protein, vegetables" for chicken soup).

Nutrition_planner.ipynb

Iteration 3:

- Snack Swap Game

To support the Snack Swap Game (Epic 7.0), we developed a refined dataset using nutrition information from the Australian Food Composition Database. This process involved filtering, grouping, and enriching food data to build game rounds suitable for young users.

1. **Merge and Standardize Datasets**

   We imported three data sources: Solids.xlsx (solid foods), Liquids.xlsx (drinks and beverages), Food_file.xlsx (food metadata). Column names were standardized to the same column names for merge, and duplicate food entries (based on Public Food Key) were removed from the liquid dataset before

joining. Each item was tagged with a Food_Type label: "Solid" or "Liquid". We selected 3 key nutritional attributes from each entry:Fat (g), Sugar (g), Sodium (mg). These datasets were merged into one combined nutrition table.

2. **Cleaning and Filtering**

   We kept only entries where the food had a 100% analysed portion and dropped unnecessary metadata columns “'Food Profile ID', 'Derivation', 'Food Description', 'Sampling Details', 'Nitrogen Factor', 'Fat Factor', 'Specific Gravity', 'Analysed Portion', 'Unanalysed Portion'”. Foods were then filtered using a list of relevant keywords in the Classification Name ("bacon", "berry", "bread", "breakfast cereal", "burger", "cabbage", "cake", "carrot", "chocolate","citrus", "cordial", "corn chip", "crumpet", "custard", "dairy", "dessert", "doughnut", "drink","egg", "energy drink", "fruit", "gelatine", "ice cream", "jam", "lolly", "milk", "muesli", "muffin", "noodle", "nut", "pasta", "peanut", "pizza", "popcorn", "porridge", "potato","preserved", "sandwich", "sausage", "scone", "slice", "soft drink", "soup", "soy", "sprout","sugar", "sweet", "water", "yoghurt") to ensure relevance and appropriateness for children.

3. **Grouping and Health Classification**

   To group similar food variants, we extracted a simplified Base_Food_Name using the first one or two parts of the full name (e.g., "Milk, chocolate, full-fat" to "Milk, chocolate").

   Then we averaged the nutrient values for each base food, applied rules to classify foods as Healthy or Unhealthy.
   A food was labelled Unhealthy if: Fat > 17.5g, or Sugar > 22.5g, or Sodium > 600mg

   We also applied manual keyword overrides to correct for known exceptions. Unhealthy included: **"cordial", "soft drink", "jelly", "fruit drink", "macaroni & cheese", "slice"**. Healthy included:
   "apple", "apricot", "date", "fig", "goji berry", "oats".

4. **Create Snack Swap Game Rounds**

   We randomly generated 50 game rounds, each containing, 1 Healthy food and 2 Unhealthy foods. The options were shuffled, and each round included, the correct answer and a simple explanation of why that choice is healthier (Less salty , less greasy, less sweet).These explanations were dynamically created by comparing the healthy option’s fat, sugar, and sodium levels to the others.The final dataset was saved with the following fields: Round, Option_1, Option_2, Option_3, Correct Answer, Why, image_1, image_2, image_3

Link:https://drive.google.com/file/d/11kRMKwRhCy2SkQGXkw6Areb6KNJEcA7Z/view?usp=drive_link

- **AI food recipe reverse**

To train a YOLO model on the Food-101 dataset, we first converted the original image classification dataset into the object detection format required by YOLO. This process involved creating a directory structure with separate images/train, images/val, labels/train, and labels/val folders. Images were divided into training and validation sets based on the official Food-101 split. For each image, we created a corresponding label file in YOLO format, assuming that the entire image represents a single object centered at (0.5, 0.5) with full width and height (1.0, 1.0). A food101.yaml configuration file was also generated, specifying the dataset structure and class names. The file structure ensures that the YOLO training script can directly read the image-label pairs and cache them efficiently for model training. The final format is fully compatible with Ultralytics YOLOv8, and supports both supervised training and image inference pipelines.

https://drive.google.com/file/d/145T5cwcAp2XSHJVLGvLFFTafz8Gj6-80/view?usp=drive_link